

Topics 38 & 39 — FastAPI Setup + Ollama Integration

Part 1: environment & dependencies. Part 2: connecting FastAPI to Ollama.



Complete Beginner



Intermediate



Expert



Interview Q&A



Layer 1 — Complete beginner

What is FastAPI, in one line



One line

FastAPI lets you write Python functions that the outside world can call over the internet.

You write a normal function, add one line above it, and now a browser, a mobile app, or another program can run it by visiting a URL.

```
@app.get("/hello")
def say_hello():
    return {"message": "Hi!"}
```

Someone visits `http://localhost:8000/hello` → your function runs → they get back `{"message": "Hi!"}`.

Why you need it at all

Right now your setup is:

Browser → OpenWebUI → Ollama

OpenWebUI is **someone else's app**. You can't add your own logic to it. What if you want to:

- Check the user is logged in before answering
- Log every question to a database
- Add your company's documents to the prompt automatically
- Charge per request
- Block certain topics

You need **your own program in the middle**:

Browser / App → YOUR FastAPI → Ollama

That middle piece is what you build. FastAPI is the tool for building it.

The words you'll keep hearing

Word	Plain meaning
API	A program other programs can call
Endpoint	One URL that does one thing (<code>/chat</code> , <code>/health</code>)
Request	Someone asking your API to do something
Response	What you send back
Server	The program that sits running, waiting for requests
GET	"Give me something" (reading)
POST	"Here's some data, do something with it" (writing)

✅ If you've used `fetch()` in JavaScript

You've been the *caller*. Now you're writing **the thing being called**.

Step 1 — Do you have Python?

```
python --version
```

If you see `Python 3.12.x` or higher, good. If you see "command not found," try `python3 --version` instead.

💡 Why python vs python3

On Windows, `python` usually works. On Mac/Linux it's often `python3` — because `python` historically meant the very old Python 2, and those systems keep the explicit name to avoid ambiguity.

⚠️ Windows install gotcha

Download from **python.org** (3.12+) and tick **"Add Python to PATH"** during install. Miss it and nothing works from the terminal.

Step 2 — Understanding virtual environments

The problem: You have two projects. Project A needs FastAPI 0.100. Project B needs 0.115. If you install packages "globally" — onto your whole computer — installing one breaks the other. There's only one slot.

The solution: Give each project **its own private folder of packages**. That folder is a **virtual environment** (`venv`).

```
your-computer/
├── project-a/
│   └── FastAPI 0.100 lives here
├── project-b/
│   └── FastAPI 0.115 lives here
└── .venv/
    └── .venv/
```

Neither can see the other

✅ If you know Node

A **venv** is **node_modules**. Same idea, same reason. Nothing shared, nothing conflicts, delete the project and the packages go with it.

The rule

Never install Python packages without a virtual environment active. Modern Python will actually refuse and show an "externally-managed-environment" error if you try.

Step 3 — Create the project

Two ways. **Way A** is the classic method every tutorial uses — learn it so you understand what's happening. **Way B** is the modern tool — then use that.

Way A — the classic way (understand this)

```
mkdir ollama-api
cd ollama-api

# Create the virtual environment
python -m venv .venv
```

`python -m venv .venv` means: "run Python's built-in `venv` tool, and make a folder called `.venv`."

Now **activate** it:

```
# Windows (CMD)
.venv\Scripts\activate

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# Mac / Linux
source .venv/bin/activate
```

Your prompt changes to show `(.venv)` at the front:

```
(.venv) C:\Users\you\ollama-api>
```

✅ That prefix is your signal

Prefix showing = it's on. No prefix = not active = don't install anything.

```
pip install fastapi uvicorn requests

# Save what you installed so others can reproduce it
pip freeze > requirements.txt
```

Later, someone else runs `pip install -r requirements.txt` and gets your exact setup. When you're done working: `deactivate`.

Way B — the modern way (use this)

`uv` does all of the above, much faster, and handles the venv for you automatically.

```
# Install uv once — Windows PowerShell
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Mac / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
uv init ollama-api
cd ollama-api

uv add fastapi uvicorn requests

uv run uvicorn main:app --reload
```

Difference	Why it matters
No manual venv creation	<code>uv</code> makes one automatically
No activation step	<code>uv run</code> handles it
<code>pyproject.toml</code>	A proper config file, like <code>package.json</code> — not a flat text list

The Node comparison

Python (classic)	Python (uv)	Node
<code>python -m venv .venv</code>	<i>(automatic)</i>	<i>(automatic)</i>
<code>pip install fastapi</code>	<code>uv add fastapi</code>	<code>npm install express</code>
<code>requirements.txt</code>	<code>pyproject.toml</code>	<code>package.json</code>
<i>(none)</i>	<code>uv.lock</code>	<code>package-lock.json</code>
<code>python main.py</code>	<code>uv run main.py</code>	<code>node index.js</code>

Step 4 — What you just installed

Package	Job
fastapi	The framework — lets you define endpoints
uvicorn	The server — actually listens on a port and receives requests
requests	An HTTP client — lets <i>your</i> code call other APIs (like Ollama)

 **Why do you need BOTH fastapi and uvicorn?**

FastAPI **describes** what your API does. It doesn't know how to receive network traffic. Uvicorn is the program that opens a port, accepts connections, and hands them to FastAPI.

FastAPI is the recipe. Uvicorn is the kitchen that cooks it.

In Node, Express does both jobs. In Python they're deliberately separate — so you can swap servers without changing your code.

Step 5 — Your first API

Create a file called `main.py`:

```
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/")
def read_root():
    return {"status": "alive"}

@app.get("/hello/{name}")
def greet(name: str):
    return {"message": f"Hello, {name}!"}
```

```
uvicorn main:app --reload
```

```

                uvicorn main:app --reload
                  ↑   ↑   ↑
                |   |   |
                |   |   └─ restart automatically when I save a file
                |   └─── the variable named "app" inside that file
                └────── the file main.py (no .py extension)
```

If you renamed your file to `server.py`, it'd be `uvicorn server:app`.

You'll see `Uvicorn running on http://127.0.0.1:8000`. Open your browser:

- `http://localhost:8000` → `{"status": "alive"}`
- `http://localhost:8000/hello/Akshay` → `{"message": "Hello, Akshay!"}`

Step 6 — The free feature that sells FastAPI

```
http://localhost:8000/docs
```

You get a **complete interactive documentation page** — every endpoint listed, with a "Try it out" button that actually calls your API. You wrote **zero lines of documentation**.

✓ This is why type hints matter

FastAPI generated all of it by **reading your function signatures**. That `name: str` in `def greet(name: str)` isn't decoration (Topic 19) — FastAPI reads it to generate docs, validate incoming data, convert types, and return proper error messages.

Write `age: int` and someone sends "hello" → FastAPI rejects it with a clear error **before your function ever runs**. You wrote no validation code.

Step 7 — Understanding the decorator

```
@app.get("/hello")
def greet():
    ...
```

That `@app.get("/hello")` line is a **decorator** (Topic 16). It doesn't change your function — it *registers* it: "when a GET request arrives at `/hello`, call this function."

```
@app.get("/items")      # reading data
@app.post("/items")     # creating something
@app.put("/items/1")    # updating
@app.delete("/items/1") # deleting
```

For your Ollama work you'll mostly use `@app.post` — because you're sending a prompt (data) up, not just fetching.

Step 8 — Receiving data with Pydantic

For a POST request, someone sends you JSON. How do you describe what you expect?

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    prompt: str
    model: str = "llama3.2"      # default if not provided

@app.post("/chat")
def chat(request: ChatRequest):
    return {"you_said": request.prompt, "using": request.model}
```

That's Pydantic from Topic 19. You define a class describing the shape of the data, and FastAPI reads the incoming JSON, checks every field's type, converts it into a real Python object, and returns a helpful **422 error** if anything's wrong.

If you know TypeScript

This is your interface — except it's enforced **at runtime**, not just at compile time. TS types vanish after compiling; Pydantic models actually check the data.

```
curl -X POST http://localhost:8000/chat ^
-H "Content-Type: application/json" ^
-d '{"prompt": "hello"}'
```

Or much easier — use [/docs](#) and click "Try it out."

Step 9 — The project structure

```
ollama-api/
├── .venv/
├── .env
├── requirements.txt
├── main.py
└── Dockerfile

├── .gitignore
├── secrets and config (never commit)
├── or pyproject.toml with uv
├── your app
└── for Topic 35's compose file
```

Your [.gitignore](#):

```
.venv/
__pycache__/
*.pyc
.env
```

[__pycache__](#) is Python's compiled bytecode cache — auto-generated, never committed.

Step 10 — Config through environment variables

Never hardcode URLs. From Topic 35, your Ollama URL differs depending on where the code runs.

```
pip install python-dotenv
```

Create [.env](#):

```
OLLAMA_BASE_URL=http://localhost:11434
DEFAULT_MODEL=llama3.2
```

```
import os
from dotenv import load_dotenv

load_dotenv()

OLLAMA_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
MODEL = os.getenv("DEFAULT_MODEL", "llama3.2")
```

The second argument is the fallback if the variable isn't set. Now:

Where it runs	What it reads
Locally	Reads <code>.env</code> → <code>localhost:11434</code>
In Docker	Compose sets it → <code>http://ollama:11434</code>
Same code, zero changes.	

Pinning versions properly

`pip freeze` captures everything including sub-dependencies, which gets messy. Better practice:

```
# requirements.txt — what YOU chose
fastapi==0.115.0
uvicorn[standard]==0.32.0
httpx==0.27.2
pydantic==2.9.2
python-dotenv==1.0.1
```

💡 Those square brackets

`uvicorn[standard]` — brackets mean **extras**: optional add-ons. `[standard]` pulls in performance libraries and WebSocket support. Worth having.

Separate your dev tools:

```
# requirements-dev.txt
-r requirements.txt
pytest==8.3.3
ruff==0.7.0
```

Same as `dependencies` vs `devDependencies` in `package.json`.

`requests` vs `httpx` — switch now

🚨 The problem with requests

It's **synchronous only**. Your Ollama calls take 5+ seconds each, and synchronous code **blocks the entire server** while waiting.

```
import httpx

# sync — identical to requests
r = httpx.get("http://localhost:11434/api/tags")

# async — what you actually want
async with httpx.AsyncClient() as client:
    r = await client.get("http://localhost:11434/api/tags")
```

Topic 18 (async) and Topic 21 (context managers) both show up in that second version. We use it properly in Topic 39.

Uvicorn flags that matter

```
uvicorn main:app --reload           # development
uvicorn main:app --host 0.0.0.0 --port 8000  # in Docker
uvicorn main:app --host 0.0.0.0 --workers 4   # production
```

Flag	Purpose
<code>--reload</code>	Restart on file save. Dev only — slow and unsafe in prod.

Flag	Purpose
<code>--host 0.0.0.0</code>	Required in Docker. Default <code>127.0.0.1</code> = unreachable (Topic 35).
<code>--workers 4</code>	Multiple processes for CPU parallelism — sidesteps the GIL (Topic 20).

⚠️ Never combine `--reload` and `--workers`

They conflict — the reloader can't manage multiple worker processes.

Splitting into multiple files

One `main.py` stops working past a few hundred lines:

```

└─ main.py          # creates the app, includes routers
    └─ config.py     # environment variables
        └─ models.py # Pydantic schemas
            └─ routers/
                └─ chat.py # /chat endpoints
                    └─ services/
                        └─ ollama.py # the Ollama client

```

```

# routers/chat.py
from fastapi import APIRouter

router = APIRouter(prefix="/chat", tags=["chat"])

@router.post("/")
async def chat(request: ChatRequest):
    ...

```

```

# main.py
from fastapi import FastAPI
from app.routers import chat

app = FastAPI(title="Ollama API")
app.include_router(chat.router)

```

`APIRouter` is FastAPI's version of Express's `Router`. Same concept exactly.

CORS — you'll hit this

If a React or Next.js frontend calls your API from a different port, the browser blocks it:

```

Access to fetch at 'http://localhost:8000' from origin
'http://localhost:3000' has been blocked by CORS policy

```

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

Never in production

`allow_origins=["*"]` together with `allow_credentials=True`. That's an open door — any site can make authenticated requests as your user.

The Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

# Dependencies first – layer caching (Topic 35)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `python:3.12-slim` is ~150 MB versus ~1 GB for the full image. Use `slim` unless you need to compile something.
- `--no-cache-dir` stops pip caching downloads inside the image — pure bloat there.

Lifespan — run code at startup and shutdown

You want **one** HTTP client shared across all requests, not a new one per request (which wastes connections badly).

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
import httpx

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    app.state.http = httpx.AsyncClient(timeout=120.0)
    yield
    # Shutdown
    await app.state.http.aclose()

app = FastAPI(lifespan=lifespan)
```

Topic 21 in the wild

That's `@asynccontextmanager` — the pending-PDF topic. Everything before `yield` runs at startup, everything after at shutdown. It's the cleanest real-world example of **why context managers matter**.

Dependency injection

```
from fastapi import Depends, Request

def get_http(request: Request) -> httpx.AsyncClient:
    return request.app.state.http

@app.post("/chat")
async def chat(
    body: ChatRequest,
    client: httpx.AsyncClient = Depends(get_http),
):
    ...
```

In tests, you override `get_http` with a fake and the endpoint never knows. Comparable to Nest.js DI, but built into the framework rather than bolted on.

Sync vs async endpoints — the trap

```
@app.get("/a")
def sync_endpoint():          # runs in a threadpool – fine
    time.sleep(5)

@app.get("/b")
async def async_endpoint():   # runs on the event loop – fine
    await asyncio.sleep(5)
```

```
@app.get("/c")
async def BAD():
    time.sleep(5)
# ❌ blocks EVERYTHING
```

🚨 Endpoint C is the dangerous one

A blocking call inside an `async def` **freezes the entire event loop** — every other user waits. Since Ollama calls take seconds, this matters enormously.

The rule: if the function is `async def`, every slow call inside it must be awaited. Use `httpx.AsyncClient`, not `requests`.

Response models

```
class ChatResponse(BaseModel):
    reply: str
    model: str
    tokens: int

@app.post("/chat", response_model=ChatResponse)
async def chat(body: ChatRequest) -> ChatResponse:
    ...
```

`response_model` validates what you send *out* and **strips anything not declared** — so an internal field can't leak into a public response by accident. It also feeds the auto-generated docs.

🎯 Interview Q&A

Q: Why do you need both FastAPI and Uvicorn?

They're different layers. FastAPI defines routes, validation, and serialization; Uvicorn is the ASGI server that binds a port, accepts connections, and passes them to the app. Keeping them separate means you can swap Uvicorn for Hypercorn or Granian without touching application code, and run under Gunicorn with Uvicorn workers in production.

Q: What's a virtual environment and why is it mandatory?

A per-project directory holding that project's Python interpreter and packages, isolated from the system Python and from other projects. It's mandatory because Python installs packages into a single shared location by default, so two projects needing different versions of the same library will conflict. Modern Python enforces this by refusing global installs with an externally-managed-environment error.

Q: How does FastAPI generate its API documentation?

From your Python type hints and Pydantic models. Function signatures and model definitions give it enough information to emit an OpenAPI schema, which Swagger UI at `/docs` renders. The same metadata drives request validation and response serialization — so types are functional, not documentation.

Q: Why use `httpx` instead of `requests` for calling Ollama?

`requests` is synchronous only. In an async endpoint, a blocking HTTP call freezes the whole event loop, so a five-second LLM call stalls every concurrent request. `httpx` offers the same interface with a real async client, so `await client.post(...)` yields control while waiting. It also supports HTTP/2 and streaming responses, which you need for token-by-token output.

Q: What's lifespan for?

Running setup and teardown around the application's life — opening a shared HTTP client or database pool at startup, closing it cleanly at shutdown. It's implemented as an async context manager: everything before `yield` runs on startup, everything after on shutdown. Without it you'd create a new client per request, exhausting connections under load.

Q: What happens if you call a blocking function inside an async `def` endpoint?

It blocks the event loop, which serves every concurrent request in that worker — so one five-second blocking call makes every other user wait five seconds. FastAPI handles plain `def` endpoints safely by running them in a threadpool, so the mistake is specifically mixing blocking calls into async functions. The fix is an async-native client, or `run_in_threadpool` if the library has no async version.

🔗 MEMORY HOOK

```
venv = node_modules .
```

FastAPI = the recipe. Uvicorn = the kitchen.

Type hints aren't comments — they're what **generates your docs and validates your data.**

Integrating Ollama with FastAPI & Python APIs

Now connect the API you just built to the model you're running.

Layer 1 — The simple version

The `ollama` library

There's a better option than raw HTTP — the **official Ollama Python library**. It wraps the calls so you don't write them yourself.

```
pip install fastapi uvicorn ollama
```

Compare the two approaches:

```
# Without the library – you build the HTTP request
import requests
r = requests.post("http://localhost:11434/api/chat", json={
    "model": "gemma:2b",
    "messages": [{"role": "user", "content": "hi"}],
    "stream": False
})
reply = r.json()["message"]["content"]

# With the library – same thing, less typing
from ollama import Client
client = Client(host="http://localhost:11434")
response = client.chat(model="gemma:2b", messages=[
    {"role": "user", "content": "hi"}
])
reply = response.message.content
```

Both send the identical HTTP request

The library just hides the plumbing. Underneath, it's still POST `/api/chat` to port 11434 — exactly what you saw in Topic 34.

The starter file

```
from fastapi import FastAPI, Body
from ollama import Client

app = FastAPI()

client = Client(host="http://localhost:11434")

@app.get("/")
def read_root():
    return {"Hello": "World"}

@app.get("/contact-us")
def contact_us():
    # ← unique name (see below)
```

```
return {"email": "you@example.com"}
```

```
@app.post("/chat")
def chat(message: str = Body(..., description="The Message")):
    response = client.chat(
        model="gemma:2b",
        messages=[
            {"role": "user", "content": message}
        ],
    )
    return {"response": response.message.content}
```

🔥 A bug that doesn't error

If two functions share a name (e.g. both called `read_root`), Python **silently overwrites** the first. Your routes still work — FastAPI registered them at decoration time — but it's confusing and will bite you later. **Give every function a unique name.**

Run and test it

```
uvicorn server:app --reload
```

If your file is `server.py`, it's `server:app` — not `main:app`.

Then go to <http://localhost:8000/docs> → **POST /chat** → **Try it out** → type a message → **Execute**. Or from the terminal:

```
curl -X POST http://localhost:8000/chat ^
-H "Content-Type: application/json" ^
-d "{\"Tell me a joke\""
```

Note the odd `"\"Tell me a joke\""` — a raw JSON *string*, not an object. That's a consequence of how the body is declared, which is the next point.

Understanding `Body(...)`

```
def chat(message: str = Body(..., description="The Message")):
```

Part	Meaning
<code>message: str</code>	Expect a string
<code>Body(...)</code>	Get it from the request body , not the URL
<code>...</code>	Ellipsis — Python's literal "required, no default"

💡 Why Body at all?

Without it, FastAPI sees a plain `str` parameter and assumes it's a **query parameter** — `?message=hello` in the URL. `Body(...)` tells it to look in the JSON body instead.

The better way — Pydantic

`Body()` works for one field. Once you want two or three, use a model (Topic 19):

```

from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str
    model: str = "gemma:2b"

@app.post("/chat")
def chat(body: ChatRequest):
    response = client.chat(
        model=body.model,
        messages=[{"role": "user", "content": body.message}],
    )
    return {"response": response.message.content}

```

Now the request is a proper JSON object, and the model is switchable per request instead of hardcoded:

```
{ "message": "Tell me a joke", "model": "gemma:2b" }
```

What `messages` actually is

```

messages=[
    {"role": "user", "content": message}
]

```

This is the **ChatML format** from Phase 4 (Topic 32). Three roles exist:

Role	Purpose
<code>system</code>	Instructions that shape all replies
<code>user</code>	What the human said
<code>assistant</code>	What the model previously said

```

messages=[
    {"role": "system", "content": "You are a terse Python tutor. Answer in one sentence."},
    {"role": "user", "content": body.message},
]

```

One line, and the personality changes completely. This is the API version of the Modelfile `SYSTEM` block from Topic 34.

The memory problem

```

You: My name is Akshay
Bot: Nice to meet you, Akshay!

You: What's my name?
Bot: I don't have access to that information.

```

The model has no memory

Every call is **completely independent**. It only knows what's in the messages array you send *this time*.

To give it memory, you send the whole conversation back each time:

```

conversation = []

@app.post("/chat")
def chat(body: ChatRequest):
    conversation.append({"role": "user", "content": body.message})

    response = client.chat(model=body.model, messages=conversation)
    reply = response.message.content

    conversation.append({"role": "assistant", "content": reply})

    return {"response": reply}

```

⚠️ Fine for learning, wrong for anything real

This global list means **all users share one conversation**, and it vanishes on restart. Layer 2 fixes it.

🌿 Layer 2 — Making it correct

Fix 1 — Per-user conversations

```

conversations: dict[str, list[dict]] = {}

class ChatRequest(BaseModel):
    session_id: str
    message: str
    model: str = "gemma:2b"

@app.post("/chat")
def chat(body: ChatRequest):
    history = conversations.setdefault(
        body.session_id,
        [{"role": "system", "content": "You are a helpful assistant."}],
    )

    history.append({"role": "user", "content": body.message})

    response = client.chat(model=body.model, messages=history)
    reply = response.message.content

    history.append({"role": "assistant", "content": reply})

    return {"response": reply, "session_id": body.session_id}

```

`setdefault` returns the existing list, or creates one with the system prompt if this session is new.

🔒 Security habit worth keeping

In a real app the session ID must come from a **verified server-side session or token**, never from a value the client sends. Otherwise anyone can pass someone else's ID and read their conversation.

Fix 2 — Async (the important one)

Synchronous code freezes your entire server for the 2–10 seconds an LLM reply takes. Two users at once means the second waits for the first to finish completely.

```

from ollama import AsyncClient

client = AsyncClient(host="http://localhost:11434")

@app.post("/chat")
async def chat(body: ChatRequest):          # ← async def
    response = await client.chat(           # ← await
        model=body.model,
        messages=history,
    )
    return {"response": response.message.content}

```

Three changes: `AsyncClient`, `async def`, and `await`. That's it. Now Python handles other requests while waiting on Ollama — Topic 18 doing real work.

The trap

Writing `async def` but calling the **sync** `Client` is *worse* than not using `async` at all — you block the event loop and freeze everything.

Sync client → **def**. **Async client** → **async def + await**. All three, or none.

Fix 3 — Error handling

```

from fastapi import HTTPException
import httpx

@app.post("/chat")
async def chat(body: ChatRequest):
    try:
        response = await client.chat(model=body.model, messages=history)
        return {"response": response.message.content}

    except httpx.ConnectError:
        raise HTTPException(503, "Ollama is not reachable. Is it running?")

    except Exception as e:
        raise HTTPException(500, f"Chat failed: {e}")

```

`HTTPException` returns a proper status code and JSON error instead of crashing with a stack trace.

Fix 4 — A health endpoint

```

@app.get("/health")
async def health():
    try:
        models = await client.list()
        return {
            "status": "ok",
            "models": [m.model for m in models.models],
        }
    except Exception as e:
        raise HTTPException(503, f"Ollama unreachable: {e}")

```

One URL that answers *"is everything wired up?"* — and lists your models, so you can confirm `gemma:2b` is actually there. Docker health checks use this too (Topic 35).

Fix 5 — Config, not hardcoded URLs

```
import os
from dotenv import load_dotenv

load_dotenv()

OLLAMA_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
DEFAULT_MODEL = os.getenv("DEFAULT_MODEL", "gemma:2b")

client = AsyncClient(host=OLLAMA_URL)
```

Situation	Value
FastAPI on your laptop	http://localhost:11434
FastAPI in Docker, Ollama in Docker	http://ollama:11434
FastAPI in Docker, Ollama native	http://host.docker.internal:11434

The same three URLs you met in OpenWebUI's settings. Hardcode it and your Docker build breaks.

The complete working file

```
import os

from dotenv import load_dotenv
from fastapi import FastAPI, HTTPException
from ollama import AsyncClient
from pydantic import BaseModel

load_dotenv()

OLLAMA_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
DEFAULT_MODEL = os.getenv("DEFAULT_MODEL", "gemma:2b")

app = FastAPI(title="Ollama API")
client = AsyncClient(host=OLLAMA_URL)

conversations: dict[str, list[dict]] = {}

SYSTEM_PROMPT = "You are a helpful assistant. Be concise."

class ChatRequest(BaseModel):
    session_id: str
    message: str
    model: str = DEFAULT_MODEL

class ChatResponse(BaseModel):
    response: str
    session_id: str

@app.get("/health")
async def health():
    try:
        result = await client.list()
        return {"status": "ok", "models": [m.model for m in result.models]}
    except Exception as e:
        raise HTTPException(503, f"Ollama unreachable: {e}")

@app.post("/chat", response_model=ChatResponse)
async def chat(body: ChatRequest):
    history = conversations.setdefault(
        body.session_id,
        [{"role": "system", "content": SYSTEM_PROMPT}],
    )
    history.append({"role": "user", "content": body.message})

    try:
        result = await client.chat(model=body.model, messages=history)
    except Exception as e:
        history.pop()  # don't keep a message that failed
        raise HTTPException(503, f"Chat failed: {e}")

    reply = result.message.content
    history.append({"role": "assistant", "content": reply})

    return ChatResponse(response=reply, session_id=body.session_id)
```

```
@app.delete("/chat/{session_id}")
async def clear(session_id: str):
    conversations.pop(session_id, None)
    return {"cleared": session_id}
```

✅ Why `history.pop()` on failure matters

Otherwise a failed request leaves an **unanswered user message** in the history, and the next call sends a malformed conversation.

🌳 Layer 3 — Streaming and structure

Streaming — why ChatGPT feels fast

ChatGPT isn't faster than your API. It just **shows you words as they're generated** instead of waiting for the full answer.

```
from fastapi.responses import StreamingResponse
import json

@app.post("/chat/stream")
async def chat_stream(body: ChatRequest):
    history = conversations.setdefault(
        body.session_id,
        [{"role": "system", "content": SYSTEM_PROMPT}],
    )
    history.append({"role": "user", "content": body.message})

    async def generate():
        full = ""
        async for chunk in await client.chat(
            model=body.model,
            messages=history,
            stream=True,          # ← the key change
        ):
            token = chunk.message.content
            full += token
            yield f"data: {json.dumps({'token': token})}\n\n"

        history.append({"role": "assistant", "content": full})
        yield "data: [DONE]\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")
```

🔗 Three earlier topics converge here

`async for` — **Topic 18** · `yield` making this a generator — **Topic 17** · `data: ... \n\n` — **Server-Sent Events**, from your JavaScript request/response deep-dive.

Consuming it from React:

```
const res = await fetch("http://localhost:8000/chat/stream", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ session_id: "abc", message: "Hello" }),
});
```

```

const reader = res.body.getReader();
const decoder = new TextDecoder();

while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    const text = decoder.decode(value);
    // parse "data: {...}" lines and append tokens to state
}

```

Structured output — JSON instead of prose

```

class Sentiment(BaseModel):
    sentiment: str
    confidence: float
    reason: str

@app.post("/analyze", response_model=Sentiment)
async def analyze(text: str = Body(..., embed=True)):
    result = await client.chat(
        model=DEFAULT_MODEL,
        messages=[
            {"role": "system", "content": "Analyse sentiment. Reply with JSON only."},
            {"role": "user", "content": text},
        ],
        format="json",  # ← forces valid JSON output
    )
    return Sentiment.model_validate_json(result.message.content)

```

`format="json"` constrains the model to emit valid JSON. Pydantic then validates it against your schema — so if the model returns nonsense, you get a clear validation error rather than a crash three functions later. This is **Topic 26 (Structured Outputs)** running in real code.

Context window management

Conversation history grows forever. Every model has a token limit — exceed it and the model silently drops your system prompt, or errors.

```

MAX_TURNS = 10

def trim(history: list[dict]) -> list[dict]:
    """Keep the system prompt plus the last N exchanges."""
    system = history[0]
    recent = history[-(MAX_TURNS * 2):]
    return [system] + [m for m in recent if m["role"] != "system"]

```

Keep the system message pinned, drop the oldest turns. More sophisticated approaches *summarise* dropped history instead of discarding it.

Shared client via lifespan

```

from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.ollama = AsyncClient(host=OLLAMA_URL)

```

```
yield
# cleanup here if needed
```

```
app = FastAPI(lifespan=lifespan)
```

The Docker piece

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
api:
  build: ./api
  ports:
    - "8000:8000"
  environment:
    - OLLAMA_BASE_URL=http://ollama:11434 # service name, not localhost
    - DEFAULT_MODEL=gemma:2b
  depends_on:
    - ollama
```

Note `server:app` matching your filename, and `--host 0.0.0.0` — or the container is unreachable.

Interview Q&A — Ollama Integration

Q: Why does an LLM chat API need you to send the whole conversation every time?

LLM inference is stateless — the model has no memory between calls and only sees the messages array in the current request. Conversational continuity is entirely the application's job: you store the history, append each turn, and resend it. This is why context window management matters, since history grows unbounded and eventually exceeds the model's token limit.

Q: Your `/chat` endpoint is `async def` but uses the synchronous client. What happens?

It blocks the event loop. FastAPI runs plain `def` endpoints in a threadpool so blocking is contained, but a blocking call inside `async def` freezes the whole loop — every concurrent request stalls for the full duration of the LLM call, which could be ten seconds. The fix is `AsyncClient` with `await`, or reverting the endpoint to plain `def`.

Q: How would you implement streaming responses?

Pass `stream=True` to the Ollama client, which returns an async iterator of chunks. Wrap consumption in an async generator that yields Server-Sent Events formatted as `data: { ... }\n\n`, and return it via FastAPI's `StreamingResponse` with media type `text/event-stream`. The frontend reads it with the Fetch API's stream reader or `EventSource`. Accumulate tokens server-side as you stream so you can append the complete reply to history at the end.

Q: How do you get reliable JSON out of an LLM?

Two layers. Constrain generation with Ollama's `format="json"` parameter, which restricts sampling to valid JSON tokens, and describe the expected schema in the system prompt. Then validate the result against a Pydantic model with `model_validate_json`, so malformed output fails loudly at the boundary instead of propagating. For production you'd add a retry that feeds the validation error back to the model.

Q: Why store the Ollama URL in an environment variable?

Because it differs by deployment context — `localhost:11434` when running the API natively, `http://ollama:11434` container-to-container in Compose, `host.docker.internal:11434` from a container to a native Ollama. Externalising it means the same image runs unchanged in all three, and it's the same mechanism that lets you swap to a hosted OpenAI-compatible endpoint later by changing only config.

🔮 MEMORY HOOK — PART 2

The model remembers nothing. Your `messages` array IS the memory.

`AsyncClient` + `async def` + `await` — all three, or none.